

Exam workflow

- Read Q: identify classes, responsibilities, data; sketch class names & fields.
- Start with data model (POJOs), constructors, getters/setters, toString().
- Implement behavior (methods) + small test in main().
- Edge cases: nulls, empty lists, negative numbers, duplicates.
- Keep code compiling: add one class at a time; run often.

Class template

```
class ClassName {
    // fields
    private int n;
    private String s;

    // constructors
    public ClassName() { this.n = 0; this.s = ""; }
    public ClassName(int n, String s) { this.n = n; this.s = s; }

    // getters/setters
    public int getN() { return n; }
    public void setN(int n) { this.n = n; }

    public String getS() { return s; }
    public void setS(String s) { this.s = s; }

    // toString
    public String toString() { return "ClassName("+n+", "+s+")"; }
}
```

Common signatures

- main: public static void main(String[] args)
- equals: public boolean equals(Object o)
- compareTo: public int compareTo(Type o) // if implementing Comparable
- hashCode: public int hashCode()

Equality / compare templates

```
public boolean equals(Object o){
    if(this==o) return true;
    if(o==null || getClass() != o.getClass()) return false;
    ClassName that = (ClassName) o;
    return n==that.n && (s==null ? that.s==null : s.equals(that.s));
}

public int compareTo(ClassName o){
    // Comparable<ClassName>
    if(n!=o.n) return n < o.n ? -1 : 1;
    if(s==null && o.s==null) return 0;
    if(s==null) return -1;
    if(o.s==null) return 1;
    return s.compareTo(o.s);
}
```

Useful patterns

Null-safe getter

```
public String getSafe() {
    return (s == null) ? "" : s;
}
```

Simple factory

```
public static ClassName of(int n, String s) { return new ClassName(n,s); }
```

Singleton (lazy)

```
class Sng {
    private static Sng inst;
    private Sng() {}
    public static Sng get(){
        if(inst==null) inst=new Sng();
        return inst;
    }
}
```

Collections w/o imports (use arrays and String methods)

- length: arr.length for arrays, s.length() for strings
- iterate: for(int i=0;i<arr.length;i++) /* arr[i] */
- resize (manual): create new array and copy
- search: linear scan
- split: s.split("regex") is allowed (String method)

String handy methods

- s.equals(t), s.equalsIgnoreCase(t)
- s.startsWith("x"), s.endsWith("y")
- s.indexOf("sub"), s.lastIndexOf("sub")
- s.contains("sub") — check substring presence
- s.charAt(i) — character at index (watch bounds)
- s.split("regex") returns String[] — beware regex special chars
- String.format("%d", x) — static formatter
- s.substring(a,b), s.substring(a)
- s.replace("a","b"), s.toLowerCase(), s.toUpperCase(), s.trim()
- String.valueOf(x), Integer.parseInt(s) (throws NumberFormatException)
- Long.parseLong(s), Double.parseDouble(s) (same parse caveats)
- s.isEmpty() and (s==null || s.isEmpty())
- Use try/catch for parse:

```
try {
    int v = Integer.parseInt(s);
} catch (NumberFormatException e) { /* fallback */ }
```

OOP design — common violations and quick fixes

- God object: one class does too much. Fix: split responsibilities into smaller classes (SRP).
- Leaky encapsulation: exposing internals (public fields, exposing arrays). Fix: make fields private; return copies for mutable arrays.
- Tight coupling: classes depend on concrete implementations. Fix: depend on abstractions (small interfaces) and pass collaborators via constructor.
- Feature envy: method uses more of another class's data than its own. Fix: move method to the class with the data or add small accessors.
- Improper inheritance: using inheritance for code reuse. Fix: prefer composition; inherit only when "is-a" relationship holds.
- Lack of immutability where beneficial: mutable shared objects causing bugs. Fix: make objects immutable (final fields, no setters) where appropriate.
- Missing equals/hashCode: failing to override equals but using objects in sets/maps. Fix: implement both equals(Object) and hashCode() consistently.
- Long parameter lists: hard to read and error-prone. Fix: introduce small value object or builder pattern.
- Silent exception swallowing: catch(Exception e) hiding errors. Fix: log or rethrow with meaningful message; handle expected exceptions explicitly.

- Overuse of static: global state leads to testability issues. Fix: use instance fields and dependency injection where possible.

I/O tips (no imports)

- Use System.out.println(...) for output; System.out.print for no newline.
- Read input via Scanner usually but imports disallowed — in that case exam likely supplies signature or helper methods; if not, use java.io classes only if allowed.
- For safe parsing: try Integer.parseInt(s) catch(NumberFormatException e) /* fallback */

Handy utilities (common methods)

- Math.abs(x), Math.max(a,b), Math.min(a,b), Math.pow(a,b), Math.sqrt(x), Math.random()
- StringBuilder sb = new StringBuilder(); sb.append(...); sb.toString()
- System.arraycopy(src,0,dest,0,src.length) for fast array copy
- Arrays.sort(arr) requires import — if disallowed implement simple sort (e.g., selection sort)
- Character.isDigit(c), Character.isLetter(c), Character.toLowerCase(c), Character.toUpperCase(c)
- Integer.toString(i), Long.toString(l), Double.toString(d)
- Object.clone() only for Cloneable; prefer copy constructors

More useful methods (no imports required)

- Safe abs without Math: int abs(int x){ return (x<0) ? -x : x; }
- beware Integer.MIN_VALUE overflow
- Clamp value: int clamp(int v,int lo,int hi) if(v<lo) return lo; if(v>hi) return hi; return v;
- Safe divide: double safeDiv(double a,double b,double def) return (b==0) ? def : (a/b);
- Integer power (fast integer pow):

```
int ipow(int a,int e){ int r=1; while(e>0){ if((e&1)==1) r*=a; a*=a; e>>=1; } return r; }
```

- Null-safe equals: boolean eq(Object a,Object b){ return a==null? b==null : a.equals(b); }
- Join strings (no String.join):

```
String join(String sep,String[] arr){
    StringBuilder sb=new StringBuilder();
    for(int i=0;i<arr.length;i++){
        if(i>0) sb.append(sep);
        sb.append(arr[i]);
    }
    return sb.toString();
}
```

- Insertion sort (stable, good for small arrays):

```
for(int i = 1; i < n; i++) {
    int v = a[i], j = i;
    while( j > 0 && a[j-1] > v){
        a[j] = a[j - 1];
        j--;
    }
    a[j] = v;
}
```

- Binary search (iterative) on sorted int[]:

```
int bsearch(int[] a, int key){
    int lo = 0, hi = a.length - 1;
    while(lo <= hi){
        int mid = (lo + hi) >> 1;
        if(a[mid] == key) return mid;
    }
}
```

```

        if(a[mid] < key) lo = mid + 1;
        else hi = mid - 1;
    }
    return -1;
}

```

- Array copy manual (if System.arraycopy unavailable):

```
for(int i=0;i<src.length;i++) dest[i]=src[i];
```

- Safe parse with default: `int parseIntOr(String s,int d)` try return `Integer.parseInt(s);catch(NumberFormatException e)return d;`
- Regex via String: `s.matches("regex"), s.replaceAll(...), s.replaceFirst(...)`
- String formatting: `String.format("%d: %s", i, s)`
- GCD/LCM:

```

int gcd(int a,int b){
    while(b!=0){
        int t = a % b;
        a = b;
        b = t;
    }
    return a<0? -a : a;
}
int lcm(int a,int b){ return a/gcd(a,b)*b; }

```

- Palindrome (string):

```

boolean isPal(String t){
    for(int i=0,j=t.length()-1;i<j;i++,j--)
        if(t.charAt(i)!=t.charAt(j)) return false;
    return true;
}

```

- Swap in array:

```

void swap(int[] a, int i, int j){
    int t = a[i];
    a[i] = a[j];
    a[j] = t;
}

```

Generics & wildcards (compact)

- Generic class: `class Box<T> private T v; public Box(T v)this.v=v; public T get()return v;`
- Method generics: `public static <T> T id(T x) return x;`
- Bounded types: `<T extends Number>` (upper bound); `<T super Integer>` not allowed in declaration, use wildcards in usage
- Wildcards: `List<? extends Number>` read-only (producer); `List<? super Integer>` write-friendly (consumer)
- PECS mnemonic: Producer Extends, Consumer Super
- Generic array warning: cannot create new `T[]`; use `Object[]` cast or `ArrayList<T>` (but imports prohibited) or pass in factory.
- Raw types (`List` instead of `List<String>`) compile with warnings — avoid where possible.
- Example: `void copy(List<? super T> dest, List<? extends T> src) for(T x: src) dest.add(x);`

Generics — examples

```

// Generic class
class Box<T> {
    private T v;
    public Box(T v){ this.v = v; }
    public T get(){ return v; }
}

```

```

    public void set(T v){ this.v = v; }
}

```

```

// Diamond operator (type inference on constructor)
Box<String> b1 = new Box<>("hello"); // diamond
Box<Integer> b2 = new Box<Integer>(42); // explicit

```

```

// Generic method (inferred and explicit type witness)
public static <T> T id(T x){ return x; }
String s = id("hi"); // inferred
Integer n = MyClass.<Integer>id(5); // explicit

```

```

// Bounded type parameter
public static <T extends Number> double sum(T a, T b){
    return a.doubleValue() + b.doubleValue();
}
double tot = sum(1, 2); // works with boxed ints

```

Algorithms snippets

Array utilities (manual)

```

int sum(int[] a) {
    int s = 0;
    for (int i = 0 ; i < a.length ; i++) {
        s+=a[i];
    }
    return s;
}

```

```

double avg(int[] a) {
    return a.length == 0 ? 0.0 : (double) sum(a) / a.length;
}

```

```

int minIndex(int[] a){
    int mi=0;
    for(int i = 1; i < a.length ; i++) {
        if (a[i] < a[mi]) mi=i;
    }
    return mi;
}

```

```

int[] copyOf(int[] a,int n){
    int[] b = new int[n];
    int m = a.length < n ? a.length : n;
    for(int i=0 ; i < m; i++) {
        b[i] = a[i];
    }
    return b;
}

```

Access modifiers quick ref

- public: visible everywhere; protected: package + subclasses; (default): package only; private: within class.
- static: belongs to class (shared); final: cannot reassign; final class: cannot extend; final method: cannot override.
- override rules: `@Override` (annotation optional), same signature, covariant return allowed, cannot reduce visibility.

OOP design quick tips

- Single responsibility: each class has one role.
- Encapsulate fields (private) and expose minimal API.
- Prefer composition over inheritance unless clearly is-a.
- Use small helper methods to keep main methods readable.

Debug checklist

- Does it compile? fix first compilation error then re-run.
- Off-by-one in loops? check indices and loop bounds.
- NullPointerException: check which var is null; add guards.
- ArrayIndexOutOfBoundsException: check `arr.length` and indices.

- Wrong output: trace with `System.out.println` intermediate values.

Java style rules (quick ref)

Identifiers

- One variable per declaration.
- Arrays are to be declared `A[] arr`
- Class modifiers: public protected private abstract default static final transient volatile synchronized native strictfp
- Class names written in UpperCamelCase
- Method names written in lowerCamelCase
- Constants (static final) in ALL_CAPS_SNAKE_CASE
- **Statements**
 - Switch statements need a default case
 - One annotation per line
 - Always use `@Override`
 - Caught exceptions should not be ignored.
 - No wildcard imports. `import java.util.*`
 - Import statements should be in alphabetical order.

Exam strategy

- Build minimal working solution first (happy path), then add error handling.
- Keep code simple and well-named; markers reward clarity.
- Comment only when necessary; short inline comments ok.

Vim quick config (relative numbers + mouse)

- Persistent: add to `~/.vimrc` (Vim) or `~/.config/nvim/init.vim` (Neovim):

```

set number           " show absolute line number on current line
set relativenumber   " show relative numbers for easier jumps
set mouse=a          " enable mouse in all modes

```

- One-off in a session: `:set nu rnu (on), :set nonu norelativenumber (off), :set mouse=a (on), :set mouse= (off).`
- Toggle relative quickly: `:set invrelativenumber`

Vim essentials — motions, search, replace

- Save/quit: `:w, :q, :wq`, force quit `:q!`
- Insert: `i` (before), `a` (after), `o/O` new line; Undo `u`, Redo `Ctrl-r`
- Move: `h j k l`, word `w e b`, line start/end `0 \$, file gg/G`
- Numbers: with relative numbers, `5j` down 5, `10dd` delete 10 lines
- Search: `/pattern` forward, `?pattern` back, `next n`, `prev N`
- Replace current line: `:s/old/new/` first match; add `g` for all in line: `:s/old/new/g`
- Replace whole file: `:%s/old/new/g` (all), add `c` to confirm: `:%s/old/new/gc`
- Word-boundary: `:\s/\textbackslash <old\textbackslash >/new/g`
- Case-insensitive: prefix pattern with `\textbackslash c` e.g., `:\s/\textbackslash cold/new/g`; or `:set ignorecase smartcase`
- Escape slashes or pick another delimiter: `:%s##path/old##path/new##g`
- Visual range replace: select, then `:s/old/new/g`

Java String.format(%...) guide

- Basic: `String.format("Name: %s, Age: %d", name, age)`
- Common conversions: `%s` string, `%d` int, `%f` float/double, `%b` boolean, `%c` char, `%x` hex, `%n` newline
- Width/precision: `%8d` pad left to width 8; `%.2f` 2 decimals; `%8.2f` width 8 with 2 decimals
- Flags: `-` left align (e.g., `%-8s`); `0` zero pad numbers (e.g., `%05d`); `+` show sign (`%+d`); space for leading space (`%d`); `,` grouping (`%,d`); `(` negatives in parentheses (`%(d)`)
- Indexing args: `%2$s,%d` uses 2nd arg with grouping; reuse: `%1$s%d %1$%x`
- Date/time (via `Formatter`): `%tY-%tm-%td` year-month-day from a `java.util.Date` or `Calendar`

```

String.format("%s scored %d/%d (%.1f%)", name, got, total, got*100.0/total)
String.format("Hex: 0x%08X", n)
String.format("%-10s | %8.2f", item, price)
String.format("%,d items", count)

```